

Objetivos da aula:

- Compreender o conceito de padrões arquiteturais e sua importância na organização de sistemas de software.
- Entender a separação de responsabilidades entre interface, lógica de negócio e dados.
- Conhecer os padrões MVC, MVP e MVVM, identificando suas características principais.
- Comparar esses padrões quanto ao fluxo de controle, acoplamento e atualização da interface.
- Relacionar esses padrões com aplicações modernas, especialmente em interfaces baseadas em estado e reatividade.

Índice:

- i. Introdução aos Padrões Arquiteturais
- ii. Comparação entre MVC, MVP e MVVM

i. Introdução aos Padrões Arquiteturais

1.1. Introdução

Nas aulas anteriores foram estudados diversos padrões de projeto orientados a objetos, organizados nas três categorias clássicas da Gang of Four:

- padrões criacionais
- padrões estruturais
- padrões comportamentais

Esses padrões ajudam a resolver problemas relacionados à:

- criação de objetos
- organização de classes
- comunicação entre objetos

Entretanto, em aplicações maiores, surge um novo tipo de preocupação: **a organização geral da aplicação.**

À medida que os sistemas se tornam mais complexos, torna-se necessário definir **como diferentes partes do software serão organizadas em níveis mais amplos**, como por exemplo:

- interface com o usuário
- lógica de negócio
- acesso a dados
- comunicação com serviços externos

Essa organização de alto nível é tratada pelos **padrões arquiteturais**.

1.2. O que são Padrões Arquiteturais

Um padrão arquitetural é uma solução que define:

- como os principais componentes de um sistema são organizados
- como esses componentes se comunicam entre si

Diferentemente dos padrões de projeto tradicionais:

Tipo de padrão	Nível de atuação
GoF (Factory, Observer, etc.)	classes e objetos
Arquiteturais	sistema como um todo

Os padrões arquiteturais ajudam a responder perguntas como:

- Como separar interface e lógica de negócio?
- Como organizar o fluxo de dados na aplicação?
- Como reduzir o acoplamento entre partes do sistema?
- Como tornar o sistema mais fácil de manter e evoluir?

1.3. Motivação: Problema Sem Arquitetura

Considere uma aplicação simples de cadastro de usuários.

```
class SistemaUsuarios {  
    cadastrar(nome: string) {  
        console.log("Recebendo entrada do usuário");  
  
        console.log("Salvando no banco:", nome);  
  
        console.log("Atualizando tela:", nome);  
    }  
}
```

Nesse exemplo, uma única classe é responsável por:

- entrada de dados
- lógica da aplicação
- persistência
- interface

Isso gera problemas como:

- baixo nível de organização
- dificuldade de manutenção
- alta dependência entre partes do sistema
- dificuldade de testes

Esse tipo de estrutura não escala para sistemas maiores.

1.4. Separação de Responsabilidades

Para resolver esse problema, é necessário separar o sistema em partes com responsabilidades bem definidas.

Uma divisão comum é:

Camada	Responsabilidade
Interface	interação com o usuário
Lógica de negócio	regras do sistema
Dados	armazenamento e recuperação

Essa separação permite:

- maior organização
- menor acoplamento
- maior reutilização
- facilidade de testes

Esse princípio está diretamente relacionado a conceitos já estudados:

- alta coesão
- baixo acoplamento

1.5. Diferentes Formas de Organizar um Sistema

Existem várias maneiras de organizar essas responsabilidades.

Entre os padrões arquiteturais mais conhecidos estão:

Padrão	Ideia principal
MVC (Model-View-Controller)	separação entre dados, interface e controle
MVP (Model-View-Presenter)	interface desacoplada da lógica
MVVM (Model-View-ViewModel)	interface baseada em estado e reatividade

Todos esses padrões tentam resolver o mesmo problema:

organizar a aplicação de forma clara e desacoplada

Entretanto, cada um faz isso de maneira diferente.

1.6. O Que Realmente Muda Entre os Padrões

Embora os nomes sejam semelhantes, a principal diferença entre MVC, MVP e MVVM está em três aspectos:

- quem controla o fluxo da aplicação
- quem conhece quem (acoplamento)
- como a interface é atualizada

De forma simplificada:

No MVC, a tela "olha" para o modelo.

No MVP, a tela "fala" com o presenter.

No MVVM, a tela "escuta" o estado do viewmodel.

Essa diferença não é evidente apenas olhando nomes ou classes — ela aparece no **fluxo de execução do sistema**.

ii. Comparação entre MVC, MVP e MVVM

2.1. Introdução

Os padrões arquiteturais MVC, MVP e MVVM possuem o mesmo objetivo:

- organizar o sistema
- separar responsabilidades
- reduzir acoplamento

Entretanto, a principal diferença entre eles está em:

- quem controla o fluxo
- como os dados chegam até a interface
- como a interface é atualizada

Para tornar essas diferenças claras, será utilizado um **mesmo problema resolvido de três formas diferentes**, conforme implementado no projeto <https://github.com/arleysouza/tpll-mvc-mvp-mvvm>.

2.2. Problema Base

Sistema de cadastro de usuários:

- o usuário realiza uma ação de cadastro
- o sistema salva os dados
- a lista de usuários é exibida no console

2.3. MVC (Model-View-Controller)

Ideia central

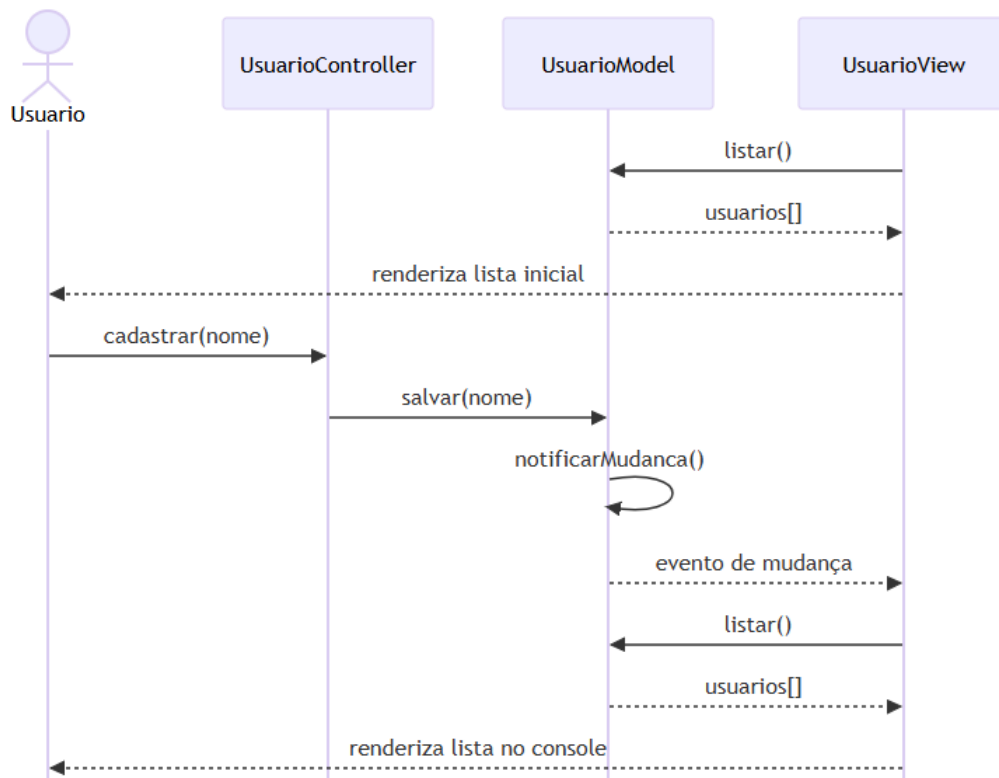
A View observa o Model e pode consultá-lo diretamente.

Funcionamento no projeto

No arquivo mvc.ts:

- a **View é criada com o Model**
- a View já renderiza o estado inicial
- o Controller recebe ações do usuário
- o Controller altera o Model
- o Model notifica mudanças
- a View consulta o Model e renderiza

Diagrama de sequência (MVC)



Interpretação didática

- a View conhece o Model
- a View observa mudanças no Model
- a View decide quando renderizar
- o Controller apenas altera o Model

✓ Ponto-chave:

A View consulta diretamente o Model.

Implicação

- simples e direto
- porém com maior acoplamento

2.4. MVP (Model-View-Presenter)

Ideia central

A View é passiva e depende do Presenter para tudo.

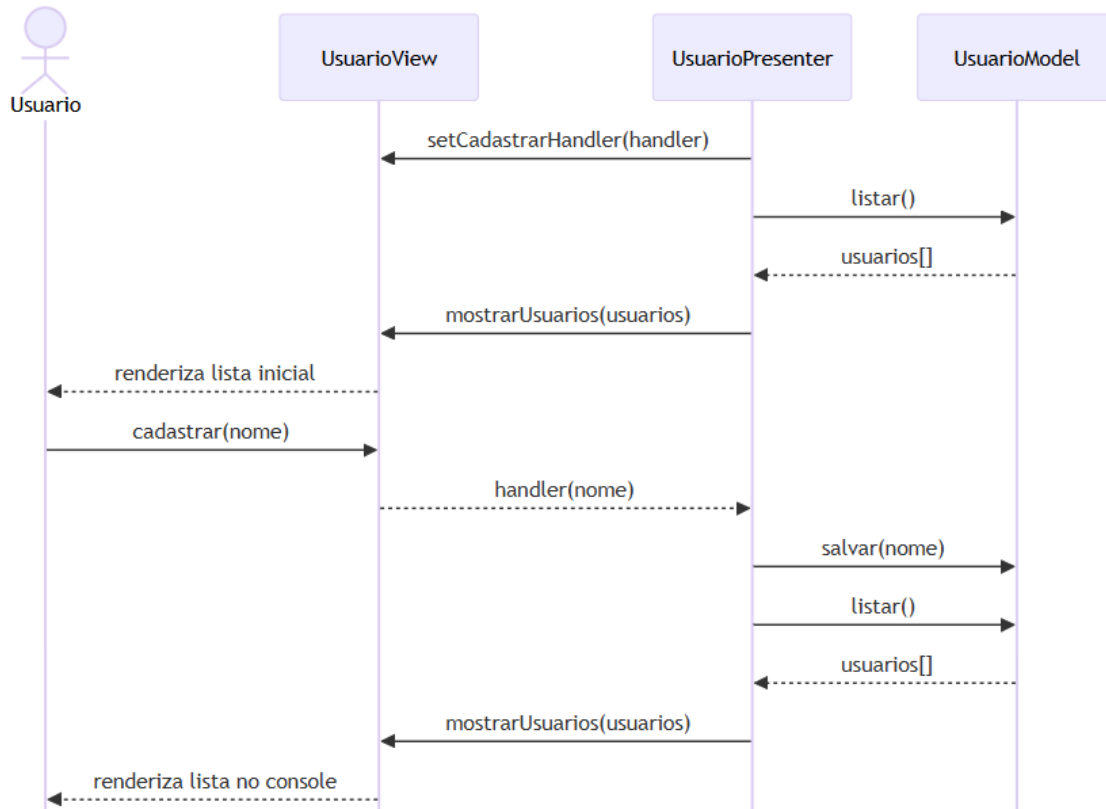
Funcionamento no projeto

No arquivo mvp.ts:

- a View **não conhece o Model**
- a View apenas dispara ações (cadastrar)
- o Presenter registra um handler na View

- o Presenter altera o Model
- o Presenter busca os dados
- o Presenter envia os dados para a View

Diagrama de sequência (MVP)



Interpretação didática

- a View não acessa o Model
 - a View apenas dispara eventos
 - o Presenter controla toda a lógica
 - o Presenter envia dados prontos
- ✓ Ponto-chave:
A View depende completamente do Presenter.

Implicação

- menor acoplamento
- maior controle
- mais fácil de testar

2.5. MVVM (Model-View-ViewModel)

Ideia central

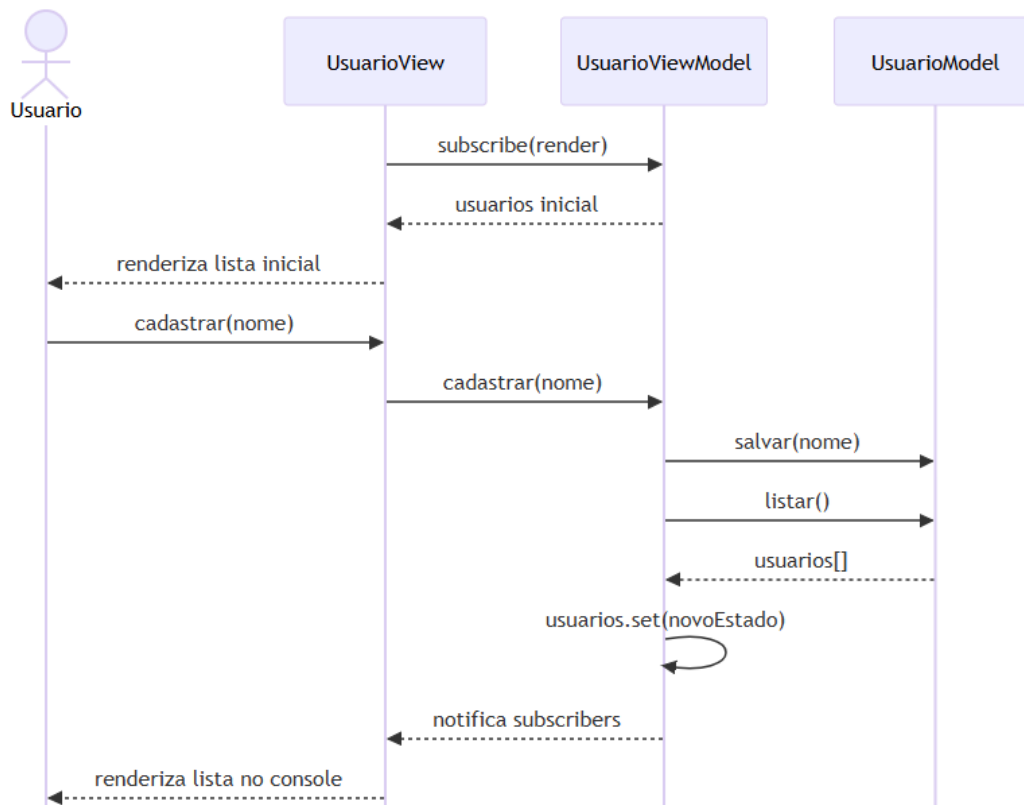
A View reage automaticamente ao estado do ViewModel.

Funcionamento no projeto

No arquivo mvvm.ts:

- a View chama o ViewModel
- o ViewModel altera o Model
- o ViewModel atualiza um estado observável
- a View está inscrita nesse estado
- a View renderiza automaticamente

Diagrama de sequência (MVVM)



Interpretação didática

- o ViewModel mantém o estado
- a View observa esse estado
- não há chamada direta de renderização
- atualização automática

✓ Ponto-chave:

A View reage ao estado do ViewModel.

Implicação

- ideal para interfaces modernas
- forte desacoplamento
- maior complexidade inicial

2.6. Comparação Direta

Quem controla o fluxo?

Padrão	Controle
MVC	Controller
MVP	Presenter
MVVM	ViewModel

A View acessa o Model?

Padrão	Acesso
MVC	✓ Sim
MVP	✗ Não
MVVM	✗ Não

Como a View é atualizada?

Padrão	Atualização
MVC	consulta o Model
MVP	chamada do Presenter
MVVM	reação ao estado

Papel da View

Padrão	Atualização
MVC	ativa + observadora
MVP	passiva
MVVM	reativa

Exercícios

Exercício 1 – Identificação do fluxo

Marque V (verdadeiro) ou F (falso) para cada afirmativa.

- () No MVC, o Controller recebe a ação do usuário e altera o Model.
- () No MVC, a View não pode acessar o Model diretamente.
- () No MVC, o Model notifica a View quando ocorre uma mudança.
- () No MVC, a View consulta o Model para obter os dados a serem exibidos.
- () No MVP, a View acessa diretamente o Model para buscar dados.
- () No MVP, o Presenter contém a lógica da aplicação.
- () No MVP, a View apenas dispara eventos e não contém lógica de negócio.
- () No MVP, o Presenter é responsável por atualizar a View.
- () No MVVM, a View chama diretamente o Model para salvar dados.
- () No MVVM, o ViewModel altera o Model.

- k) () No MVVM, a View reage automaticamente às mudanças de estado.
- l) () No MVVM, o estado da interface está no ViewModel.

Exercício 2 – Análise conceitual

Responda:

1. Qual padrão apresenta maior acoplamento entre View e Model?
2. Qual padrão facilita mais a realização de testes?
3. Qual padrão é mais adequado para interfaces reativas modernas?

Exercício 3 – Identificação de padrão

Considere o código a seguir:

```
class Tela {  
    constructor(private estado: GerenciadorEstado) {  
        this.estado.dados.subscribe(this.render.bind(this));  
    }  
  
    render(valores: string[]) {  
        console.log(valores);  
    }  
}  
  
class GerenciadorEstado {  
    dados = new ObservableValue<string[]>([]);  
  
    adicionar(nome: string) {  
        this.dados.set([...this.dados]);  
    }  
}
```

Pergunta:

- a) Qual padrão arquitetural está sendo utilizado?
- b) Justifique sua resposta com base no fluxo da aplicação.

Exercício 4 – Refatoração

Considere o seguinte cenário:

Em um sistema, a View acessa diretamente o Model para buscar dados e renderizar.

- a) Esse sistema está mais próximo de qual padrão?
- b) O que deve ser modificado para transformá-lo em MVP?

Exercício 5 – Análise arquitetural

Explique por que o React, mesmo não sendo um framework MVVM clássico, utiliza conceitos semelhantes a esse padrão.